Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs

Anonymous Submission
Author details omitted for double-blind review

Abstract*—Speculative decoding is a promising way to accelerate local LLM inference, yet its behavior on consumer-class RTX GPUs remains under-characterized. We present a controlled study on one RTX4090 with a Qwen2.5-3B-Instruct target and same-family 0.5B/1.5B drafts across GSM8K, a 5-subject MMLU subset, and CNN/DailyMail, using frozen manifests and matched prompts. We evaluate 12 fixed-policy configurations ($k \in \{4, 8, 16\}$, deterministic/stochastic, two draft sizes), regime-matched autoregressive baselines, and an Adaptive Cascaded Speculative Decoding (ACSD) policy. The best observed speedup is 0.5B, k=4, deterministic (S=3.0674, 95% CI $[3.029, 3.104]$, $p < 10^{-4}$). Speedup decreases monotonically with larger k ($\bar{S}_{k=4} = 2.9164$, $\bar{S}_{k=8} = 2.4029$, $\bar{S}_{k=16} = 1.8019$) even as B_{eff} increases, indicating a drafting/verification overhead inflection on this hardware profile. Deterministic decoding is faster in 5 of 6 matched fixed-policy draft–k pairs ($\Delta S_{\text{det-stoch}} = 0.0568$ mean). ACSD also remains strongly faster than autoregressive baselines (S=2.9561 deterministic, S=2.8658 stochastic; both $p < 10^{-4}$) with low fallback rates (1.6% and 2.2%), but does not surpass the best fixed k=4 point. These results provide deployment-oriented guidance for selecting fixed versus adaptive speculative control on consumer GPUs.

***Index Terms*—speculative decoding, large language models, inference acceleration, benchmarking, reproducibility**

I. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits its single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length-k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [1], [2].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on datacenter hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

This direction matters for privacy-sensitive and latency-critical local use, including local workstations and edge-side decision-support workflows. We therefore position this paper as a hardware-aware characterization study, rather than a new decoding algorithm.

Within this scope, ACSD is not introduced as a mechanism to exceed the peak throughput of the best static speculative setting. Instead, we position ACSD as a hardware-aware tail-latency guardrail for dynamically difficult inputs, trading a small amount of mean throughput for more controlled runtime behavior under acceptance drops.

A. Research questions

We address three research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding delivers against autoregressive decoding on a single RTX4090 with a 3 B target, and how close is this to a practical hardware-constrained upper bound?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency, and what do these trends imply about memory-traffic and kernel-launch overhead on consumer GPUs?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

B. Contributions

This paper makes the following contributions:

1) A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two decoding regimes across GSM8K, MMLU subset, and CNN/DailyMail.
2) A complete 12-configuration speculative grid (two draft sizes, $k \in \{4, 8, 16\}$, deterministic/stochastic) with regime-matched baselines and per-configuration reporting of latency, throughput, acceptance, and quality deltas.
3) Empirical evidence that, in the current RTX4090 CUDA-graph run family, speedup decreases with larger k (highest at k=4), with best observed performance at 0.5B, k=4, deterministic (S=3.0674), and a deterministic advantage in 5 of 6 matched draft–k pairs.
4) A statistical reporting protocol for camera-ready evaluation: paired bootstrap confidence intervals, one-sided significance tests for S>1, and corrected pairwise comparisons for winner selection.
5) An adaptive-policy design guidance and an ACSD quantitative validation on the same manifests.

The rest of the paper is organized as follows. Section II reviews related work. Section III defines the experimental protocol; Section IV defines metrics. Section V describes implementation and reproducibility controls. Section VI reports the main empirical findings. Section VII outlines an adaptive speculative-strategy framework and its observed runtime behavior. Section VIII discusses implications, limitations, and conclusions.

II. Related Work

a) Vanilla speculative decoding: Leviathan *et al.* [1] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [2] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \qquad (1)$$

where $\tau \in [1, k{+}1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [3] earlier explored blockwise parallel decoding within a single model.

b) Reducing drafting cost: Medusa [4] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [5]–[7] keeps an external draft but conditions it on target hidden features, reporting substantial speedups under their settings. Despite these refinements, T_{draft} remains linear in k, forcing EAGLE-3 to a single transformer layer and capping reported gains at about 2–3$\times$ on strong GPUs.

c) Diffusion-based drafters: Recent work directly attacks the linear-in-k drafting bottleneck. DiffuSpec [8] and SpecDiff-2 [9] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [10] focuses on a low-cost parallel draft model adaptation. DFlash [11] uses a lightweight 5-layer block-diffusion drafter conditioned on target hidden features and reports up to $5\times$ speedup over autoregressive baselines on H200/B200 hardware.

d) Cascaded speculative control: Recent advancements in cascaded architectures attempt to dynamically balance inference efficiency and generation quality. For instance, CAS-Spec constructs a hierarchy of draft stages on-the-fly from a single target model using Dynamically Switchable Inference Acceleration (DSIA) strategies, such as layer sparsity and activation quantization, coordinated by a Dynamic Tree Cascade (DyTC) algorithm [12]. Similarly, Speculative Cascades employs a hybrid approach with a deferral rule, invoking larger, costlier models only for complex inputs while using smaller models for simpler tasks, thereby optimizing the cost-quality trade-off [13]. While these state-of-the-art methods achieve substantial acceleration, they inherently require either surgical modifications to the target model's internal architecture (e.g., dynamic sparsity) or complex external serving infrastructure to handle deferral logic. In stark contrast, our proposed ACSD operates as a strictly training-free and architecture-agnostic online controller. By dynamically scheduling between physically independent, same-family draft models (0.5B and 1.5B) based on real-time acceptance heuristics, ACSD circumvents the massive overhead of intra-model modifications and new drafter training. Its primary design objective is not to compete for theoretical peak throughput on data-center accelerators, but to serve as a robust, hardware-aware tail-latency guardrail that guarantees runtime stability on resource-constrained consumer-class GPUs.

e) Precision and systems deployment: Deployment-oriented inference work shows that quantization and memory traffic are central to practical throughput gains on constrained hardware. Low-precision techniques such as LLM.int8() [14] reduce weight movement and can interact with speculative decoding in two opposing ways: lower per-step cost for draft/verify passes, but possible shifts in acceptance behavior when verifier and drafter precision settings differ. This interaction motivates reporting speed and quality jointly, rather than throughput alone.

f) Distributed and edge relevance: In distributed and mobile computing settings, local inference can reduce round-trip latency and cloud dependency for interactive workloads. In this context, speculative decoding is relevant beyond chatbot serving, including edge language interfaces where bounded latency and predictable runtime matter as much as peak throughput.

g) Position of this work: This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen2.5-3B target, 0.5B/1.5B drafts) under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, we characterize practical operating points on a single RTX4090, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to evaluate adaptive cascaded speculative decoding (ACSD) as a runtime-stability extension under the same hardware budget.

III. Experimental Protocol

A. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1) **GSM8K** [15] test subset: 300 samples.
2) **MMLU** [16] subset: 500 samples (5 subjects $\times$ 100 each).
3) **CNN/DailyMail** [17] subset: 200 samples.

Sampling policy: Fixed random seed $= 42$, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. Each output CSV records the seed and sample identifier, enabling paired analysis at the sample level.

B. Prompt set

We use the following fixed prompt templates for each task:

TABLE I
HARDWARE PROFILE USED FOR THE RTX4090 RUN FAMILY.

Component	Specification
GPU	NVIDIA RTX4090 GPU
GPU memory	24 GB GDDR6X
Memory bandwidth	1008 GB/s
CUDA cores	16384
Architecture	Ada Lovelace
Power profile	TGP class 450 W
Software stack	PyTorch + Transformers (pinned env)

GSM8K:

You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
Question: {question}

MMLU:

You are a knowledgeable assistant. Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:

CNN/DailyMail:

Summarize the following article in 3–4 sentences, preserving key facts and entities.
Article: {article}
Summary:

a) Objective and hypotheses: The study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from k=4 to k=8 improves speedup, while k=16 may reduce gains due to additional drafting and verification overhead.
- **H3:** A larger draft (1.5B) achieves higher acceptance and higher net speed than a smaller draft (0.5B).
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k.

b) Hardware and software controls: All experiments run on one NVIDIA RTX4090 (24GB VRAM) with a fixed software stack (PyTorch, Transformers, and pinned tokenizer settings). We use the same runtime entry points and prompt templates for all configurations, and run serially to avoid cross-run contention.

c) CUDA-graph runtime setup: To reduce per-step launch overhead, we use a CUDA-graph-capable verify path. For each k, we keep fixed step shapes, pre-allocate static draft/verify buffers, and warm up before optional `torch.cuda.CUDAGraph` capture. During decoding, each cycle updates buffer contents and replays the graph when shape guards hold; otherwise, it falls back to eager execution. This preserves output semantics while reducing host-side overhead in short speculative cycles.

d) Model pairing: Target model: Qwen2.5-3B-Instruct. Draft models: Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. All models are from the same family to minimize tokenizer mismatch and style drift. Each configuration processes the same fixed 1,000-prompt manifest.

e) Task-entropy interpretation: We treat the three tasks as distinct decoding-entropy regimes to interpret acceptance behavior: GSM8K is a lower-entropy chain reasoning with concentrated next-token mass; MMLU is medium-entropy constrained QA; CNN/DailyMail is higher-entropy open-ended summarization. This framing is used only for interpretation of acceptance and quality trends, not as an additional optimization objective.

f) Decoding regimes and grid: Two regimes are evaluated:

- **Deterministic:** greedy decoding (T=0.0, top-p=1.0; sampling disabled).
- **Stochastic:** sampling with T=0.7, top-p=0.9, and no top-k truncation.

For each draft model, we sweep $k \in \{4, 8, 16\}$, producing 12 speculative configurations. We also run one autoregressive baseline per regime for anchored latency and throughput reporting.

g) Endpoints: The primary endpoint is paired wall-clock speedup $S = L_{\text{AR}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft–k–regime combination. Secondary endpoints include acceptance rate α, effective accepted block size B_{eff}, mean per-sample latency, throughput (tokens/s), and task-level quality deltas. Camera-ready statistical reporting uses paired bootstrap confidence intervals (10,000 resamples) and one-sided tests for S>1, with corrected pairwise winner tests.

IV. EVALUATION METRICS

Table II defines the exact reporting metrics used in this study.

A. Success criteria

Primary:

1) Mean speedup $S \geq 1.3\times$ with CI_S lower bound > 1.0.
2) Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (draft size, k) maximizing S while satisfying quality constraints, and report pairwise significance for winner selection with multiple-testing correction.

V. IMPLEMENTATION AND REPRODUCIBILITY

a) Code organization: The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, and autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

b) Model loading and precision: In the current configuration, target and draft models are loaded in FP16 for the main sweep, with quantization controls retained in configuration for alternative runs. The tokenizer and generation limits are kept constant between baselines and speculative runs to preserve paired comparability.

TABLE II
EXACT METRIC DEFINITIONS FOR REPORTING.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup confidence interval	CI_S	paired bootstrap percentile CI of S (10k resamples)	×	Higher lower-bound
Speedup significance	p_S	one-sided paired test for $H_0 : S \leq 1$	p-value	Lower
Speedup stability	σ_S	std(S over seeds)	×	Lower

c) Baselines: We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

d) Speculative decoding implementation.: The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Figure 1 summarizes this implementation flow.

e) CUDA-graph overhead model and observed effect: For a speculative cycle, we model runtime as

$$T_{\text{cycle}} = k\, t_{\text{draft}} + t_{\text{verify}} + n_{\text{launch}}\, t_{\text{launch}}, \tag{2}$$

where $n_{\text{launch}}\, t_{\text{launch}}$ captures host-side kernel-launch overhead. With successful graph replay, the launch cost is reduced to a lower replay overhead term:

$$T^{\text{graph}}_{\text{cycle}} \approx k\, t_{\text{draft}} + t_{\text{verify}} + n_{\text{replay}}\, t_{\text{replay}}, \qquad t_{\text{replay}} \ll t_{\text{launch}}. \tag{3}$$

In the present RTX4090 run family, capture-rate metadata is 0, so the main results reflect fallback execution rather than sustained graph replay. A separate paired GPU-optimization toggle microbenchmark (12 matched samples, k=8) reports mean TPOT 32.84 ms/token (off) versus 33.30 ms/token (on), with mean delta $+0.46$ ms/token (95% CI $[0.14, 0.83]$), indicating no TPOT benefits under the current capture conditions.

f) Dynamic-scheduling bottleneck in native PyTorch paths: Fallback eager execution should not be viewed only as an implementation miss. It reflects a broader systems challenge for end-to-end dynamic speculation. When acceptance-dependent control flow, sampling branches, and cache updates change per step, preserving lossless semantics often requires CPU-visible orchestration between kernels. Without a deeper runtime/kernel redesign, this serial control overhead can erase expected gains from speculative parallelism, as also emphasized in modern inference runtime roadmaps for dynamic scheduling.

g) Tokenizer isomorphism as an enabling design choice: Same-family draft/target pairing is a central engineering decision in this study. Because the draft and target tokenizers are isomorphic, speculative verification avoids per-step vocabulary alignment and token translation. Cross-tokenizer pipelines introduce additional CPU-side mapping overhead that can materially reduce or negate speculative speedup on constrained hardware.

h) Regimes and outputs: The same verification core is used for deterministic and stochastic modes, with only the sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

VI. RESULTS

This section reports fixed-policy speculative-decoding outcomes from the RTX4090 CUDA-graph run family and regime-matched deterministic/stochastic baselines. Headline values are summarized in *all_configs_summary.csv*. We emphasize paired comparative metrics (S, α, B_{eff}) and task deltas to evaluate the speed–quality tradeoff.

Figure 2 complements Table III with a visual baseline-relative view across all configurations, including speedup, latency/throughput deltas, and token-timing deltas, and acceptance dynamics.

A. RQ1: Does speculative decoding accelerate inference?

Across the RTX 4090 CUDA-graph run family, all 12 speculative configurations exceed S=1, with speedups from 1.7773 to 3.0674. The best setting is 0.5B with k=4 in deterministic mode. Regime-matched baseline files report mean latencies of 11.41s (deterministic) and 11.65s (stochastic) per sample, so the wall-clock gains are substantial for this fixed workload and prompt set. Paired bootstrap intervals and one-sided tests show that every configuration is faster than autoregressive decoding ($p_S < 10^{-4}$ throughout, Table III).

TABLE III

RTX4090 CUDA-GRAPH RUN FAMILY SUMMARY (QWEN2.5-3B TARGET). S IS THE RATIO-OF-MEANS PAIRED SPEEDUP AGAINST REGIME-MATCHED BASELINES. CI_S IS THE PAIRED BOOTSTRAP 95% INTERVAL (10K RESAMPLES OVER MATCHED SAMPLES); p_S IS ONE-SIDED SIGNIFICANCE FOR $H_0: S \leq 1$. α IS TOKEN-LEVEL ACCEPTANCE; B_{EFF} IS ACCEPTED TOKENS PER VERIFICATION CYCLE.

Config	S	CI_S	p_S	α	B_{eff}	ΔGSM8K	ΔMMLU	ΔCNN/DM
baseline, det	1.0000	–	–	–	–	–	–	–
baseline, stoch	1.0000	–	–	–	–	–	–	–
0.5B, k=4, det	**3.0674**	[3.029, 3.104]	$< 10^{-4}$	0.4212	1.66	0.33	0.00	-0.02
0.5B, k=4, stoch	2.9394	[2.900, 2.979]	$< 10^{-4}$	0.4080	1.61	1.67	-0.20	-0.03
0.5B, k=8, det	2.4542	[2.408, 2.500]	$< 10^{-4}$	0.3515	2.72	0.33	0.00	-0.05
0.5B, k=8, stoch	2.4063	[2.359, 2.453]	$< 10^{-4}$	0.3444	2.67	1.00	0.00	-0.38
0.5B, k=16, det	1.8050	[1.751, 1.859]	$< 10^{-4}$	0.2539	3.76	0.33	0.00	-0.01
0.5B, k=16, stoch	1.8185	[1.764, 1.873]	$< 10^{-4}$	0.2516	3.73	0.34	-0.20	-0.19
1.5B, k=4, det	2.8726	[2.839, 2.905]	$< 10^{-4}$	0.4581	1.81	-0.33	0.00	-0.03
1.5B, k=4, stoch	2.7860	[2.754, 2.819]	$< 10^{-4}$	0.4421	1.75	1.67	0.00	-0.34
1.5B, k=8, det	2.4066	[2.366, 2.447]	$< 10^{-4}$	0.3929	3.04	0.00	0.00	-0.06
1.5B, k=8, stoch	2.3444	[2.304, 2.385]	$< 10^{-4}$	0.3822	2.96	3.67	0.20	-0.47
1.5B, k=16, det	1.8068	[1.761, 1.853]	$< 10^{-4}$	0.2968	4.39	0.67	0.00	-0.03
1.5B, k=16, stoch	1.7773	[1.732, 1.825]	$< 10^{-4}$	0.2928	4.36	1.00	-0.20	-0.36

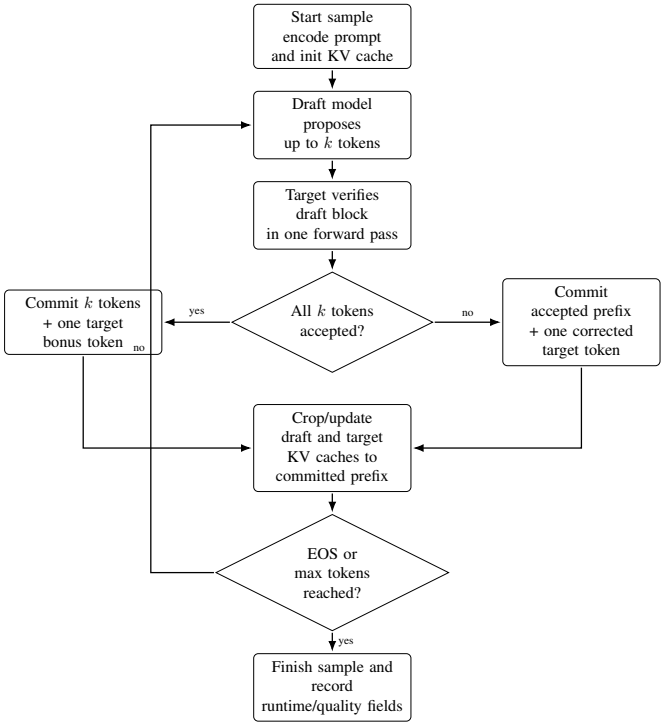

Fig. 1. Vanilla speculative decoding implementation flow. The target model remains the verifier of record; accepted draft prefixes are committed, and caches are cropped to preserve exact target-consistent decoding.

B. RQ2: How do draft length and draft size affect acceptance and speed?

Draft length shows a monotonic decline in speedup over this grid. Averaging over drafts and regimes, mean speedup is highest at k=4 (2.9164), then k=8 (2.4029), then k=16 (1.8019). This is consistent with acceptance dynamics: as k increases, B_{eff} increases (1.7075 $\rightarrow$ 2.8475 $\rightarrow$ 4.0600) but α decreases (0.4324 $\rightarrow$ 0.3678 $\rightarrow$ 0.2738), and the extra draft/verify cost dominates net benefit.

TABLE IV

DETERMINISTIC BOTTLENECK ANALYSIS AT LOW VS. HIGH k.

Config	α	B_{eff}	S	**Interpretation**
0.5B, k=4	0.4212	1.66	3.0674	Sweet spot
0.5B, k=16	0.2539	3.76	1.8050	Drafting overload
1.5B, k=4	0.4581	1.81	2.8726	Memory pressure
1.5B, k=16	0.2968	4.39	1.8068	Diminishing returns

Draft size has a smaller global effect than k in this grid, and the 0.5B draft is slightly better on mean speedup (mean S: 2.4151 for 0.5B versus 2.3323 for 1.5B).

Table IV summarizes the deterministic endpoints of this non-linear trend and highlights the systems-level interpretation: higher B_{eff} does not guarantee higher S once per-cycle drafting and verification overhead dominates.

C. RQ3: Deterministic versus stochastic regime

Deterministic decoding is faster in 5 of 6 matched draft–k pairs. The deterministic advantage ranges from -0.0135 to 0.1280 in absolute S and averages 0.0568 across the six matched pairs; the single exception is 0.5B, k=16, where stochastic is slightly faster. This still supports a preference for a deterministic deployment regime when latency is primary, and output diversity is not required.

At the system level, stochastic decoding adds sampling and rejection-check overheads that are weakly amortized on this hardware profile. With large distribution processing and limited memory bandwidth (1008 GB/s), these costs can offset speculative parallelism gains, consistent with the deterministic advantage.

Quality stability differs by task. CNN/DailyMail remains close to baseline but consistently negative (-0.47 to -0.01). MMLU shifts are small (-0.2 to 0.2). GSM8K shows the largest spread (-0.33 to 3.67), especially under stochastic decoding. Under our entropy-oriented interpretation, CNN/DailyMail is the highest-entropy setting in this suite, so

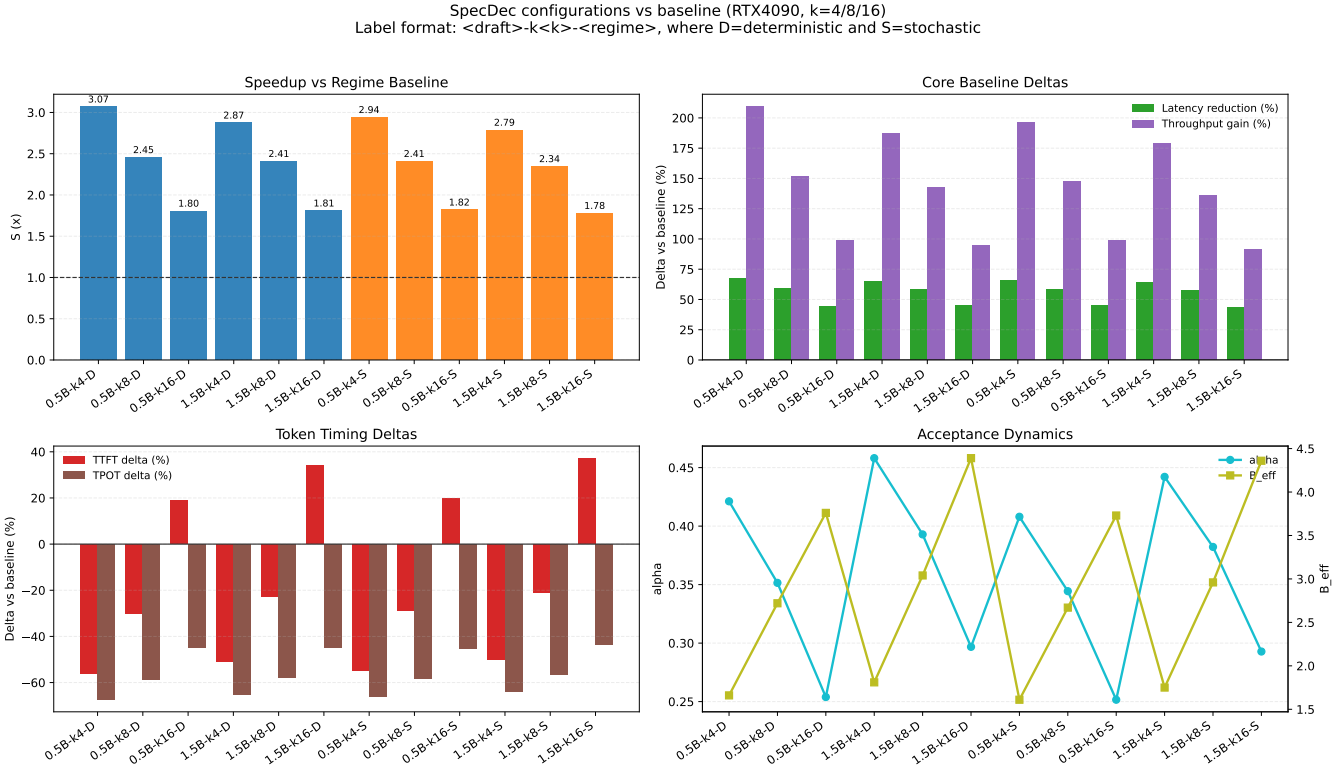

Fig. 2. Speculative-decoding metrics for all RTX 4090 configurations relative to regime-matched baseline. Top-left: paired speedup S (baseline at $S{=}1$). Top-right: mean latency reduction and throughput gain percentages vs baseline. Bottom-left: TTFT and TPOT percentage deltas vs baseline. Bottom-right: acceptance dynamics (α and B_{eff}) across the same configurations.

acceptance mismatch and quality drift are more likely than in constrained QA tasks. Under a conservative quality filter ($\max|\Delta| \leq 1$ across tasks), 9 configurations remain, with best speedup at 0.5B, $k{=}4$, deterministic.

The resulting design guideline is clear: when selecting k on consumer GPUs, prioritize lower per-cycle runtime over maximal accepted-block growth.

Implementation details for the CUDA-graph-capable runtime setup are described in Section III.

D. ACSD quantitative evaluation

We next evaluate the Adaptive Cascaded Speculative Decoding (ACSD) controller under the same 1000-sample deterministic/stochastic manifests (GSM8K 300, MMLU 500, CNN/DailyMail 200). Speedup is computed as paired ratio-of-means against regime-matched autoregressive baselines using the same bootstrap protocol as Table III.

Both ACSD regimes remain strongly faster than autoregressive decoding ($p < 10^{-4}$ for both), but do not exceed the best fixed-policy point (0.5B, $k{=}4$): ACSD deterministic is lower by 0.1113 speedup units (≈3.6%), and ACSD stochastic by 0.0736 (≈2.5%). Operationally, fallback use remains low (1.6% deterministic, 2.2% stochastic), indicating that tail guardrails are only occasionally invoked, while rescue drafting is used in 17.0% and 20.0% of samples. Task-level ACSD quality deltas are (0.33, 0.00, -0.02) for deterministic and (-1.00, -0.20, -0.22) for stochastic on (GSM8K, MMLU, CNN/DailyMail), respectively.

Aggregated adaptive-k usage shows that ACSD spends most accepted blocks at larger settings near the sweet spot boundary: deterministic run share is $k{=}4$ (40.74%) and $k{=}5$ (46.18%), with only 1.73% at $k \in \{1, 2\}$; stochastic run share is $k{=}4$ (43.17%) and $k{=}5$ (42.66%), with 1.75% at $k \in \{1, 2\}$. This pattern supports the controller's intent of staying near high-throughput operating points while retaining low-k escape routes.

Table VI clarifies the trade-off: ACSD mean latency is 2.6–3.8% above fixed $k{=}4$, with slightly higher P99 (+1.8% deterministic, +3.6% stochastic), yet far below high-k tails (about 45–47% lower P99 than $k{=}16$). This supports ACSD as a guardrail, not a peak-throughput optimizer.

E. Summary of empirical operating point

The current evidence supports a practical recommendation: use deterministic speculative decoding with a smaller fixed block ($k{=}4$) when peak throughput is the primary target, and prefers the 0.5B draft for this hardware profile. ACSD adds robustness-oriented control with strong baseline-relative speedups and low fallback rates, but fixed $k{=}4$ remains the fastest observed operating point in this run family.

TABLE V
ACSD RESULTS ON RTX 4090 CUDA-GRAPH RUN FAMILY, WITH QUALITY DELTAS RELATIVE TO REGIME-MATCHED BASELINE QUALITY ANCHORS INFERRED FROM THE FIXED-POLICY SUMMARY TABLE.

Regime	S (95% CI)	α	B_{eff}	R_{tok}	Rescue/Fallback	$\max \lvert\Delta Q\rvert$
ACSD, det	2.9561 [2.9205, 2.9916]	0.4207	1.7977	32.6056	17.0% / 1.6%	0.33
ACSD, stoch	2.8658 [2.8305, 2.9007]	0.4071	1.7214	31.0750	20.0% / 2.2%	1.00

TABLE VI
LATENCY-TAIL COMPARISON (SECONDS): FIXED 0.5B k=4, ACSD, AND HIGH-k 0.5B k=16.

Regime	**Config**	Mean	P90	P99
det	0.5B k=4	3.721	7.423	8.153
det	ACSD	3.861	7.715	8.300
det	0.5B k=16	6.323	12.688	15.654
stoch	0.5B k=4	3.963	7.855	8.655
stoch	ACSD	4.065	8.163	8.962
stoch	0.5B k=16	6.406	13.088	16.260

VII. DISCUSSION ON ADAPTIVE SPECULATIVE STRATEGIES DERIVED FROM EMPIRICAL FINDINGS

The current empirical study identifies the strongest fixed-policy speedup at a smaller block size (k=4), with net speed degrading as k increases, even when the accepted block length rises. This pattern indicates that fixed-policy drafting is vulnerable to acceptance/overhead imbalance and long-tail latency. We therefore design and evaluate an adaptive-policy variant that keeps standard target-side verification while adapting control decisions online.

ACSD is intentionally not framed as a new peak-throughput optimizer. Its role is to provide a practical tail-latency guardrail under dynamic acceptance collapse, while keeping the mean performance near the best static operating point.

Advanced variants such as EAGLE-3 and DFlash reduce drafting cost through new drafter architectures and dedicated training pipelines, often evaluated on strong accelerators. Our objective is different: to maintain a stack of drafters / verifiers of the same family that is deployable on consumer hardware, and recover speed and stability by runtime control (adaptive k, rescue switching, and tail fallback) rather than a newly trained drafter.

A. Design objective

Let speculative-cycle latency be

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \tag{4}$$

where τ is the expected number of accepted tokens per cycle. In fixed-policy speculative decoding, poorly matched samples can reduce τ and increase total verify steps, producing a heavy-tail runtime. ACSD explicitly targets this effect by adapting draft policy at the sample level.

B. Formalization of the ACSD Policy

To rigorously define the runtime behavior, we formalize the ACSD controller as a deterministic finite-state machine (FSM). Let t denote the current verification step, and k_t be the dynamic draft length. The system state is defined as $S_t \in \{\mathcal{D}_{pri}, \mathcal{D}_{res}, \mathcal{M}_{AR}\}$, representing the primary drafter (0.5B), the rescue drafter (1.5B), and the autoregressive fallback mode, respectively. The state transitions and scheduling parameters are directly governed by the real-time empirical acceptance rate $\alpha_t = \frac{\tau_t}{k_t}$, where τ_t is the number of accepted tokens at step t. ACSD incorporates four core online control mechanisms:

1) **Dynamic Draft Length (k_t):** While operating in $\mathcal{D}_{pri}$ or $\mathcal{D}_{res}$, k_t is modulated within a narrow operational neighborhood around the empirical optimum of 4 (typically 3 to 5). Specifically, the controller updates the length via $k_{t+1} = \min(k_t + 1, k_{max})$ if $\alpha_t \geq \theta_{up}$, and $k_{t+1} = \max(k_t - 1, k_{min})$ if $\alpha_t < \theta_{down}$. This bounding strategy effectively caps the memory bandwidth overhead during target-side verification.
2) **Cascaded Draft Switch ($S_t \rightarrow \mathcal{D}_{res}$):** We maintain a continuous penalty counter C_t to track consecutive low-acceptance events. If $\alpha_t < \theta_{rescue}$, the counter increments; otherwise, $C_t = 0$. Once $C_t \geq N_{rescue}$, the controller triggers a state transition to the 1.5B rescue drafter ($S_{t+1} = \mathcal{D}_{res}$) to leverage its higher representational capacity for difficult distributions. To mitigate scheduling oscillation, a cooling window W_{cool} (measured in generation steps) is strictly enforced before any subsequent model switching.
3) **Tail Latency Guardrail ($S_t \rightarrow \mathcal{M}_{AR}$):** To prevent severe tail-latency degradation under sustained high-entropy inputs, a terminal fallback mechanism is employed. If the rescue drafter fails to recover the acceptance rate (i.e., $\alpha_t < \theta_{fallback}$ for $N_{fallback}$ consecutive steps) after generating a minimum prefix length L_{min}, the system forcefully transitions to pure autoregressive decoding ($S_{t+1} = \mathcal{M}_{AR}$) for the remainder of the sample.
4) **Checkpointed Execution:** The execution context, including partial generation sequences and verification logs, is serialized at predefined intervals to guarantee system resilience and allow safe resumption of interrupted runs.

C. How this guidance is used

Future adaptive-policy evaluation should remain a strict incremental study under the same protocol, with primary comparison against fixed k=4 and high-k (k=16) operating points.

D. Observed ACSD outcomes

Measured ACSD outcomes are reported in Section VI (Table V). In brief, ACSD achieves strong acceleration rel-

ative to autoregressive baselines in both regimes (S=2.9561 deterministic, S=2.8658 stochastic; both $p < 10^{-4}$), while keeping fallback rates low (1.6% and 2.2%). However, ACSD does not exceed the fastest fixed-policy setting (0.5B, k=4 deterministic at S=3.0674).

For deployment, this should be read as a stability trade: ACSD sacrifices a small amount of mean throughput versus fixed k=4, but keeps tails close to that operating point and far from the high-k tail-latency regime (Table VI), while using rescue/fallback only when needed.

E. Scope in this paper

This section combines design rationale with measured ACSD behavior under the same protocol. Quantitative claims are restricted to the reported fixed policy and ACSD runs in this manuscript; broader generalization still requires additional hardware/model families.

VIII. Discussion

A. Main empirical takeaways

Three findings are consistent in the current run family. First, speculative decoding provides consistent acceleration across all tested configurations ($S > 1$ throughout). Second, speedup declines as k increases in this run family, with the strongest performance at k=4 and the weakest at k=16. Third, deterministic decoding outperforms stochastic decoding in 5 of 6 matched draft–k pairs.

The measured ACSD extension follows the same direction: it remains strongly faster than autoregressive decoding in both regimes, but does not exceed the best fixed-policy throughput point (0.5B, k=4, deterministic). ACSD therefore appears most useful as a robustness-oriented control layer: mean latency is slightly above fixed k=4, but tail behavior remains close to that point and well below the high-k collapse.

Together, these results suggest that practical deployment should prioritize a smaller block size (k=4) and deterministic policy when latency is the primary objective.

Demand for local and consumer-hardware LLM inference is increasing, so the key question is not only peak data-center speed, but which policy remains stable on constrained personal hardware. We therefore report hardware-profiled operating points rather than universal defaults.

B. Draft-size implications

The 0.5B draft produces the best single configuration (0.5B, k=4, deterministic), and also has a slightly higher mean speedup than 1.5B across the full grid in this run family. This indicates that draft-size selection is hardware- and budget-sensitive: the smaller draft can win on end-to-end speed even when the larger draft improves acceptance.

C. Speed versus quality

Quality drift is task-dependent. CNN/DailyMail and MMLU remain relatively stable across settings, while GSM8K is more variable, especially under stochastic decoding. A conservative filter ($\max|\Delta| \leq 1$ across all three tasks) retains a small subset of configurations, among which 1.5B, k=8, deterministic remains best. This supports a practical policy: use deterministic mode with k=4 for production-like latency targets, and treat stochastic settings as quality-sensitive configurations requiring additional calibration. Final winner claims should be interpreted together with paired confidence intervals and corrected pairwise tests from a coherent artifact family.

IX. Threats to validity

Single-hardware scope. All results are from one RTX4090. Relative rankings are informative for this deployment class, but absolute speedups may shift on other GPU generations. In this run family, CUDA-graph metadata indicates a pending/zero capture rate, so reported gains reflect the same code path under fallback execution rather than successful graph replay.

Model-family scope. All reported runs use the same family pairings (Qwen2.5 target and drafts). Cross-family pairings may therefore exhibit materially different acceptance and quality behavior.

Evaluation breadth. The benchmark set spans math reasoning, knowledge QA, and summarization, but does not exhaust safety, factuality, or long-context behavior. ROUGE-L in particular is an incomplete summarization quality proxy.

X. Conclusion

This paper provides a controlled consumer-GPU evaluation of speculative decoding with a Qwen2.5-3B target and same-family drafts. The current evidence shows consistent acceleration across all tested fixed and adaptive configurations, with best observed performance at 0.5B, k=4, deterministic (S=3.0674). We find that increasing k from 4 to 8 and 16 reduces net speedup in this setup, and that deterministic decoding is generally faster than stochastic decoding for matched settings. ACSD also delivers a significant speedup over autoregressive baselines (deterministic S=2.9561, stochastic S=2.8658) with low fallback rates, but remains below the best fixed-policy speedup.

These findings yield an actionable operating recommendation for the present hardware budget: prefer deterministic speculative decoding at k=4, with 0.5B draft as the default fast path and larger drafts reserved for quality-sensitive scenarios. ACSD can be enabled when runtime resilience and tail protection are prioritized over absolute peak throughput. Future work will evaluate broader hardware/model families and tune adaptive thresholds to test whether ACSD can close the remaining gap to the fixed k=4 optimum while preserving quality control.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2211.17192

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," in *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.01318

[3] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1811.03115

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.10774

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.15077

[6] ——, "EAGLE-2: Faster inference of language models with dynamic draft trees," *arXiv preprint arXiv:2406.16858*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2406.16858

[7] ——, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," *arXiv preprint arXiv:2503.01840*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2503.01840

[8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, "DiffuSpec: Unlocking diffusion language models for speculative decoding," *arXiv preprint arXiv:2510.02358v1*, 2025, https://arxiv.org/abs/2510.02358v1. [Online]. Available: https://doi.org/10.48550/arXiv.2510.02358

[9] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto, "SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding," *arXiv preprint arXiv:2511.00606*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.00606

[10] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum, "PARD: Accelerating llm inference with low-cost parallel draft model adaptation," *arXiv preprint arXiv:2504.18583*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2504.18583

[11] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block diffusion for flash speculative decoding," *arXiv preprint arXiv:2602.06036*, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2602.06036

[12] Z. Ning, J. Shao, R. Xu, X. Guo, J. Zhang, C. Zhang, and X. Li, "CAS-Spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms," *arXiv preprint arXiv:2510.26843*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.26843

[13] H. Narasimhan, W. Jitkrittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, "Faster cascades via speculative decoding," in *Proceedings of the IEEE Conference (arXiv preprint)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.19261

[14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2208.07339

[15] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2110.14168

[16] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2009.03300

[17] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. [Online]. Available: https://doi.org/10.18653/v1/D18-1206